

14 · Cursors and Records

Why this matters in interviews. "What is a cursor? Implicit vs explicit?", "walk me through OPEN–FETCH–CLOSE", "what is %ROWTYPE ?", "cursor FOR loop vs explicit cursor", "what does SQL%ROWCOUNT return?" — cursors are how PL/SQL processes a query result **one row at a time**.

In this chapter: the context area · implicit cursors and SQL% attributes · explicit cursors (declare, open, fetch, close) · cursor FOR loops · parameterised cursors · records (%ROWTYPE , user-defined) · choosing between them.

Sample table used throughout (CUSTOMERS):

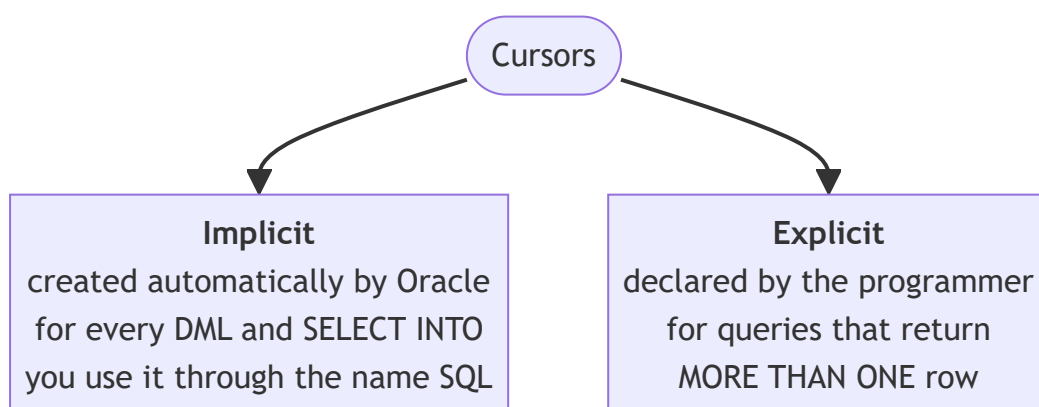
ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000
2	Khilan	25	Delhi	1500
3	kaushik	23	Kota	2000
4	Chaitali	25	Mumbai	6500
5	Hardik	27	Bhopal	8500
6	Komal	22	MP	4500

14.1 Why cursors?

When Oracle processes a SQL statement it creates a memory area called the **context area**. It holds everything needed to run the statement — including **the rows returned** (one or more) and information about the statement.

A **cursor is a pointer to this context area**. PL/SQL controls the context area through a cursor. The set of rows the cursor holds is called the **active set**. You can **name** a cursor so a program can refer to it and **fetch and process the rows one at a time**.

There are **two types**:



14.2 Implicit cursors

Whenever a **DML statement** (`INSERT` , `UPDATE` , `DELETE`) — or a `SELECT ... INTO` — is issued, an **implicit cursor is associated with it**. For `INSERT` the cursor holds the data to insert; for `UPDATE` and `DELETE` it identifies the rows that would be affected. You don't declare, open or close it — Oracle does.

In PL/SQL you can refer to the **most recent implicit cursor as the SQL cursor**, which always has these attributes:

Attribute	Returns
<code>SQL%FOUND</code>	TRUE if an <code>INSERT</code> / <code>UPDATE</code> / <code>DELETE</code> affected one or more rows , or a <code>SELECT INTO</code> returned one or more rows; otherwise FALSE
<code>SQL%NOTFOUND</code>	the opposite of <code>%FOUND</code> — TRUE if the DML affected no rows (or a <code>SELECT INTO</code> returned none)
<code>SQL%ROWCOUNT</code>	the number of rows affected by the DML statement (or returned by <code>SELECT INTO</code>)
<code>SQL%ISOPEN</code>	always FALSE for an implicit cursor — Oracle closes it automatically as soon as the statement finishes

The `SQL` cursor additionally has `%BULK_ROWCOUNT` and `%BULK_EXCEPTIONS` , designed for use with the **FORALL** statement. Any attribute is accessed as `sql%attribute_name` .

`SQL` is the **built-in implicit cursor** Oracle provides for the **most recently executed** `INSERT` , `UPDATE` , `DELETE` or `SELECT INTO` . Implicit cursors exist for **all** such statements, whatever number of rows they affect — a `DELETE` that removes 10,000 rows uses one too. (For `SELECT INTO` , exactly one row is *required*; otherwise an exception is raised.)

Example — raise every salary by 500 and report how many rows changed

```
DECLARE
    total_rows NUMBER(2);
BEGIN
    UPDATE customers
    SET     salary = salary + 500;

    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('no customers selected');
    ELSIF SQL%FOUND THEN
        total_rows := SQL%ROWCOUNT;
        DBMS_OUTPUT.PUT_LINE(total_rows || ' customers selected ');
    END IF;
END;
/
```

6 customers selected

PL/SQL procedure successfully completed.

(All six customers were updated; Hardik's salary is now **9000** — that is why the later examples show 9000.) If the table were empty `SQL%NOTFOUND` would be **TRUE** and the first message printed.

SELECT ... INTO and the implicit cursor

SELECT ... INTO must return **exactly one row**, otherwise Oracle raises an exception instead of setting SQL%NOTFOUND :

Rows returned	Result
0	NO_DATA_FOUND (ORA-01403)
1	OK — SQL%FOUND TRUE, SQL%ROWCOUNT = 1
more than 1	TOO_MANY_ROWS (ORA-01422)

So you handle the "not found" case of a SELECT INTO with an EXCEPTION section, not by testing SQL%NOTFOUND .

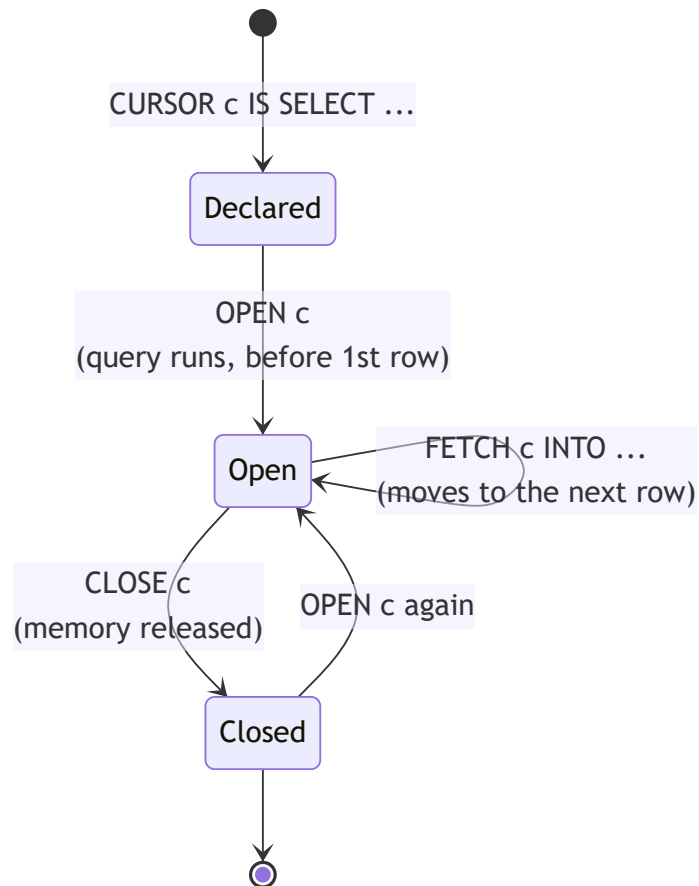
14.3 Explicit cursors

Explicit cursors are programmer-defined cursors for gaining **more control over the context area**. An explicit cursor is defined in the **declaration section** of the PL/SQL block and is created on a SELECT statement that **returns more than one row**.

```
CURSOR cursor_name IS select_statement;
```

Working with an explicit cursor is always **four steps**:

Step	Statement	What happens
1 · Declare	CURSOR c_customers IS SELECT id, name, address FROM customers;	defines the cursor: a name and its SELECT — initialises the memory
2 · Open	OPEN c_customers;	allocates the memory, runs the query , and prepares the rows; the cursor points before the first row
3 · Fetch	FETCH c_customers INTO c_id, c_name, c_addr;	moves to the next row and copies its columns into variables — one row per FETCH
4 · Close	CLOSE c_customers;	releases the memory ; you cannot fetch again unless you re-open



Analogy. You ask a library for all the books by one author. **Opening** the cursor = the librarian *collects all the matching books and puts them on a shelf for you*. **Fetching** = you *pick one book at a time* to read its details. **Closing** = you're done and the librarian *clears the shelf*, freeing the space.

The complete pattern

```

DECLARE
  c_id   customers.id%TYPE;
  c_name customers.name%TYPE;
  c_addr customers.address%TYPE;

  CURSOR c_customers IS
    SELECT id, name, address FROM customers;
BEGIN
  OPEN c_customers;                -- 1
  LOOP
    FETCH c_customers INTO c_id, c_name, c_addr; -- 2 one row per pass
    EXIT WHEN c_customers%NOTFOUND; -- no more rows → leave
    DBMS_OUTPUT.PUT_LINE(c_id || ' ' || c_name || ' ' || c_addr);
  END LOOP;
  CLOSE c_customers;              -- 3
END;
/

```

```
1 Ramesh Ahmedabad
2 Khilan Delhi
3 kaushik Kota
4 Chaitali Mumbai
5 Hardik Bhopal
6 Komal MP
```

PL/SQL procedure successfully completed.

Step by step: `OPEN` executes the query and the cursor points **before the first row**. Each `FETCH` moves to the **next row** and copies its values into `c_id`, `c_name`, `c_addr`; you usually **loop and fetch until no rows remain**. `CLOSE` releases the memory.

Place `EXIT WHEN cursor%NOTFOUND` immediately after the `FETCH`. `%NOTFOUND` becomes `TRUE` when the fetch found no row; testing it later means you would process the last row twice (with the previous values still in the variables).

Explicit-cursor attributes

Explicit cursors have the same four attributes, written `cursor_name%attribute`:

Attribute	TRUE / value when ...
<code>%ISOPEN</code>	the cursor is open
<code>%FOUND</code>	the last <code>FETCH</code> returned a row (NULL before the first fetch)
<code>%NOTFOUND</code>	the last <code>FETCH</code> returned no row
<code>%ROWCOUNT</code>	the number of rows fetched so far

Common cursor errors

Error	Cause
<code>ORA-01001: Invalid cursor (INVALID_CURSOR)</code>	fetching or closing a cursor that isn't open
<code>ORA-06511: PL/SQL: cursor already open (CURSOR_ALREADY_OPEN)</code>	<code>OPEN</code> on an open cursor
resource leak / <code>ORA-01000: maximum open cursors exceeded</code>	forgetting <code>CLOSE</code> — especially inside loops or on error paths

14.4 Three ways to write the same loop

Print every employee earning more than 30,000. Assume `emp(emp_name, salary)` holds `('Kavi', 25000)`, `('Mala', 32000)`, `('Nila', 45000)`.

Style 1 — explicit cursor, scalar variables

```
DECLARE
    CURSOR cur IS SELECT emp_name, salary FROM emp;
    v_name    VARCHAR2(50);
    v_salary  NUMBER;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_name, v_salary;
        EXIT WHEN cur%NOTFOUND;
        IF v_salary > 30000 THEN
            DBMS_OUTPUT.PUT_LINE('Name : ' || v_name || ' Salary : ' || v_salary);
        END IF;
    END LOOP;
    CLOSE cur;
END;
/
```

Style 2 — cursor `FOR` loop

Oracle **opens the cursor, fetches every row and closes it** for you, and declares the loop record automatically:

```
BEGIN
    FOR rec IN (SELECT emp_name, salary FROM emp) LOOP
        IF rec.salary > 30000 THEN
            DBMS_OUTPUT.PUT_LINE('Name : ' || rec.emp_name || ' Salary : ' || rec.salary);
        END IF;
    END LOOP;
END;
/
```

Style 3 — explicit cursor with a `%ROWTYPE` record

```
DECLARE
    CURSOR cur IS SELECT emp_name, salary FROM emp;
    rec cur%ROWTYPE;                                -- a record shaped like one row of the cursor
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO rec;
        EXIT WHEN cur%NOTFOUND;
        IF rec.salary > 30000 THEN
            DBMS_OUTPUT.PUT_LINE('Name : ' || rec.emp_name || ' Salary : ' || rec.salary);
        END IF;
    END LOOP;
    CLOSE cur;
END;
/
```

All three print the same thing:

```
Name : Mala Salary : 32000
Name : Nila Salary : 45000
```

	Style 1	Style 2 — FOR loop	Style 3
OPEN / FETCH / CLOSE	manual	automatic	manual
Variables	one per column	none — the record is implicit	one record
Code size / risk of leaks	more	least	medium
Control (e.g. stop early, reuse)	full	limited	full

Use the cursor FOR loop by default. Reach for the manual form only when you need explicit control — fetching in batches, testing `%ROWCOUNT`, or passing a cursor around.

What is `rec` in `FOR rec IN (SELECT ...)`?

`rec` is called a **record variable**.

- Each **row** returned by the query in parentheses becomes a **record**.
- The record has **fields corresponding to the columns** in the `SELECT`.

For a join query such as

```
FOR rec IN (SELECT o.ordno, o.orddate, li.itemno, li.qty, li.price, li.disrate
              FROM   orders o
              JOIN    lineitems li ON o.ordno = li.ordno
              WHERE   o.custno = p_custno) LOOP
    -- use rec.column_name here
END LOOP;
```

rec has these fields:

Field	Access
ordno	rec.ordno
orddate	rec.orddate
itemno	rec.itemno
qty	rec.qty
price	rec.price
disrate	rec.disrate

The record's field names come from the **column names or aliases** — alias any expression (salary * 12 AS annual) so it gets a usable field name.

14.5 Parameterised cursors

A cursor can take **parameters**, so one definition serves many inputs. *Employees with a salary above 30,000 in a given department* (here the classic EMP(ename, job, sal, deptno) table):

```
SET SERVEROUTPUT ON;
DECLARE
    -- Parameterised cursor: takes a department number as input
    CURSOR emp_cursor (p_dept_id NUMBER) IS
        SELECT ename, job, sal
        FROM emp
        WHERE deptno = p_dept_id
              AND sal > 30000;
BEGIN
    -- Loop for department 10
    DBMS_OUTPUT.PUT_LINE('Employees in Department 10 with salary > 30000:');
    FOR rec IN emp_cursor(10) LOOP
        DBMS_OUTPUT.PUT_LINE('Name: ' || rec.ename ||
                              ', Job: ' || rec.job ||
                              ', Salary: ' || rec.sal);
    END LOOP;

    -- Loop for department 20
    DBMS_OUTPUT.PUT_LINE(chr(10) || 'Employees in Department 20 with salary > 30000:');
    FOR rec IN emp_cursor(20) LOOP
        DBMS_OUTPUT.PUT_LINE('Name: ' || rec.ename ||
                              ', Job: ' || rec.job ||
                              ', Salary: ' || rec.sal);
    END LOOP;
END;
/
```

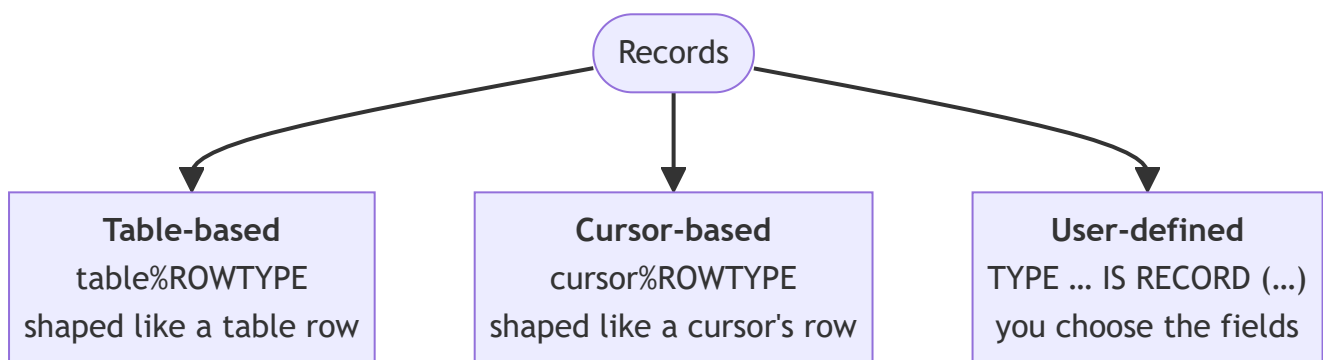
Notes:

- The **parameter** is given in the cursor's declaration; its value is supplied when the cursor is **opened** (`emp_cursor(10)`) — inside a `FOR` loop, at the loop header.
- Parameters can have **default values**: `CURSOR c (p_min NUMBER DEFAULT 0) IS ...`.
- `CHR(10)` is a **line feed** — it prints a blank line before the second heading.
- The same parameterised cursor is reused for two departments without repeating the query text.

14.6 Records

A **record** is a **data structure that can hold data items of different kinds** — its **fields** are similar to the columns of a row in a table. Imagine you track the **books in a library**: each has a *Title*, *Author*, *Subject* and *Book ID*. A record with a field for each lets you treat a **BOOK as a logical unit** and organise its information better.

PL/SQL handles **three kinds of records**:



`%ROWTYPE` — what does it mean?

```
customer_rec customers%ROWTYPE;
```

`%ROWTYPE` is a **record type attribute** in PL/SQL. It tells Oracle: "Create a variable that can hold an entire row of data from the specified table (or cursor)." — i.e. "declare a variable named `customer_rec` that has the same structure as one row in the `customers` table." If a column is added later, the record follows.

Table-based records

```
DECLARE
    customer_rec customers%ROWTYPE;
BEGIN
    SELECT * INTO customer_rec
    FROM   customers
    WHERE  id = 5;

    DBMS_OUTPUT.PUT_LINE('Customer ID: ' || customer_rec.id);
    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || customer_rec.name);
    DBMS_OUTPUT.PUT_LINE('Customer Address: ' || customer_rec.address);
    DBMS_OUTPUT.PUT_LINE('Customer Salary: ' || customer_rec.salary);
END;
/
```

```
Customer ID: 5
Customer Name: Hardik
Customer Address: Bhopal
Customer Salary: 9000
```

PL/SQL procedure successfully completed.

`SELECT *` with `%ROWTYPE` is the one place `*` is fine: the record has exactly the table's columns, in order. Access a field with the **dot operator**: `record_name.field_name`.

Cursor-based records

```
DECLARE
    CURSOR customer_cur IS
        SELECT id, name, address FROM customers;
    customer_rec customer_cur%ROWTYPE;
BEGIN
    OPEN customer_cur;
    LOOP
        FETCH customer_cur INTO customer_rec;
        EXIT WHEN customer_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(customer_rec.id || ' ' || customer_rec.name);
    END LOOP;
    CLOSE customer_cur;
END;
/
```

```
1 Ramesh
2 Khilan
3 kaushik
4 Chaitali
5 Hardik
6 Komal
```

```
PL/SQL procedure successfully completed.
```

The record has exactly the **columns the cursor selects** (`id` , `name` , `address`) — no `salary` , so `customer_rec.salary` would not compile.

User-defined records

You define the type yourself, then declare variables of it:

```
TYPE type_name IS RECORD (
    field_name1 datatype1 [NOT NULL] [:= DEFAULT_EXPRESSION],
    field_name2 datatype2 [NOT NULL] [:= DEFAULT_EXPRESSION],
    ...
    field_nameN datatypeN [NOT NULL] [:= DEFAULT_EXPRESSION]
);
record_name type_name;
```

The library-book record:

DECLARE

```
TYPE books IS RECORD (  
    title  VARCHAR2(50),  
    author VARCHAR2(50),  
    subject VARCHAR2(100),  
    book_id NUMBER
```

```
);
```

```
book1 books;
```

```
book2 books;
```

BEGIN

```
-- Book 1 specification
```

```
book1.title  := 'C Programming';
```

```
book1.author := 'Nuha Ali ';
```

```
book1.subject := 'C Programming Tutorial';
```

```
book1.book_id := 6495407;
```

```
-- Book 2 specification
```

```
book2.title  := 'Telecom Billing';
```

```
book2.author := 'Zara Ali';
```

```
book2.subject := 'Telecom Billing Tutorial';
```

```
book2.book_id := 6495700;
```

```
DBMS_OUTPUT.PUT_LINE('Book 1 title : ' || book1.title);
```

```
DBMS_OUTPUT.PUT_LINE('Book 1 author : ' || book1.author);
```

```
DBMS_OUTPUT.PUT_LINE('Book 2 title : ' || book2.title);
```

```
DBMS_OUTPUT.PUT_LINE('Book 2 book_id : ' || book2.book_id);
```

END;

/

Book 1 title : C Programming

Book 1 author : Nuha Ali

Book 2 title : Telecom Billing

Book 2 book_id : 6495700

Remember the order of declaration: define the `TYPE` first, *then* a variable of that type — a variable cannot be declared before its type exists (the lab note said exactly this):

```

DECLARE
    TYPE employee_type IS RECORD (
        emp_id    NUMBER,
        emp_name  VARCHAR2(50),
        salary    NUMBER
    );
    rec employee_type;           -- ← AFTER the TYPE
BEGIN
    rec.emp_id    := 1;
    rec.emp_name  := 'Murugan';
    rec.salary    := 200000;
    DBMS_OUTPUT.PUT_LINE('ID : ' || rec.emp_id || ' Name : ' || rec.emp_name || ' Salary : ' ||
rec.salary);
END;
/

```

```
ID : 1 Name : Murugan Salary : 200000
```

Other useful facts:

- A record **type declared in a package specification** can be shared with callers (the `test_pac.emp_record` of Chapter 13).
- Records can be passed as **parameters** and returned from **functions**.
- Whole-record assignment works between records of the **same type**: `rec2 := rec1;` . Compare fields, not whole records (`=` on records is not allowed).
- DML with records: `INSERT INTO customers VALUES customer_rec;` and `UPDATE customers SET ROW = customer_rec WHERE id = customer_rec.id;` .

14.7 Implicit vs explicit — at a glance

	Implicit cursor	Explicit cursor
Created by	Oracle, automatically	the programmer (<code>CURSOR ... IS SELECT ...</code>)
Used for	every <code>INSERT / UPDATE / DELETE</code> , and <code>SELECT INTO</code> (exactly 1 row)	queries returning 0 ... many rows
Name	<code>SQL</code>	your own
Open / fetch / close	automatic	manual (or automatic in a <code>FOR</code> loop)
Attributes	<code>SQL%FOUND</code> , <code>%NOTFOUND</code> , <code>%ROWCOUNT</code> , <code>%ISOPEN</code> (always FALSE)	<code>c%FOUND</code> , <code>%NOTFOUND</code> , <code>%ROWCOUNT</code> , <code>%ISOPEN</code> (real)
Parameters	–	yes
Control	none	full

Set-based beats row-by-row. A cursor loop processes rows one at a time ("**slow-by-slow**"). If a single `UPDATE ... WHERE` or `INSERT ... SELECT` can do the job, use it — it's far faster. Reach for cursors when each row needs procedural logic. For large volumes, `BULK COLLECT` and `FORALL` cut the overhead.

Interview corner

Q. What is a cursor? A pointer to the private SQL work area (context area) holding a statement's result; it lets PL/SQL process a multi-row result one row at a time.

Q. Implicit vs explicit cursors? Implicit: created automatically for every DML and `SELECT INTO`, accessed as `SQL`. Explicit: declared by the programmer for multi-row queries, with manual `OPEN` / `FETCH` / `CLOSE`.

Q. Steps of an explicit cursor? Declare → Open → Fetch (repeat) → Close.

Q. Cursor attributes? `%FOUND`, `%NOTFOUND`, `%ISOPEN`, `%ROWCOUNT` (plus `%BULK_ROWCOUNT` / `%BULK_EXCEPTIONS` on `SQL`).

Q. What does `SQL%ROWCOUNT` give after an `UPDATE`? The number of rows updated.

Q. What is `SQL%ISOPEN` for an implicit cursor? Always `FALSE` — Oracle closes it automatically.

Q. Cursor `FOR` loop vs explicit cursor? The `FOR` loop opens, fetches and closes automatically and declares the record implicitly — shorter and safer; explicit gives finer control.

Q. What is `%ROWTYPE`? `%TYPE`? `%TYPE` = the data type of one column/variable; `%ROWTYPE` = a record with the structure of a whole table/cursor row.

Q. What is a record? Kinds? A composite variable with named fields of different types: table-based, cursor-based, user-defined.

Q. What happens if `SELECT INTO` returns no row / many rows? `NO_DATA_FOUND` / `TOO_MANY_ROWS`.

Q. What is a parameterised cursor? A cursor whose query uses parameters supplied when it is opened, so one definition serves many inputs.

Q. Why put `EXIT WHEN c%NOTFOUND` right after `FETCH`? So the loop stops before processing a stale row from the previous fetch.

Quick recap

- **Cursor** = pointer to the context area holding a statement's rows (the **active set**).
- **Implicit** (`SQL`): automatic for DML and `SELECT INTO`; attributes `%FOUND` `%NOTFOUND` `%ROWCOUNT`, and `%ISOPEN` always `FALSE`.
- **Explicit:** `CURSOR c IS SELECT ...` → `OPEN` → `FETCH ... INTO` (loop, `EXIT WHEN c%NOTFOUND` right after) → `CLOSE`.
- **Cursor `FOR` loop** does open/fetch/close for you; `rec.column` fields come from the `SELECT` list.
Parameterised cursors: `CURSOR c (p NUMBER) IS ... WHERE x = p`.
- **Records** group fields of different types: **table-based** (`t%ROWTYPE`), **cursor-based** (`c%ROWTYPE`), **user-defined** (`TYPE ... IS RECORD`, type before variable). Access with `record.field`.
- `SELECT INTO` needs exactly one row (`NO_DATA_FOUND` / `TOO_MANY_ROWS`). Prefer set-based SQL; use cursors when each row needs logic.